INSTITUTO POLITÉCNICO NACIONAL
CENTRO DE INVESTIGACIÓN EN COMPUTACIÓN

Session-Based Recommendation using Graph Neural Networks and Attention Mechanisms

Name
Ing. Angel Emanuel Rodríguez Román

Profesors:
Dr. Luis Pastor Sánchez Fernández
Dr. Ricardo Ménchaca Méndez
Dr. Rolando Quintero Téllez
Dr. Erik Zamora Gómez Dr. Mario Eduardo Rivero

June 12th, 2026

Abstract

Sequential recommendation is a key task in recommender systems, especially in scenarios where users interact anonymously and no persistent profile is available [**?**]. This work addresses the problem of session-based recommendation, which consists of predicting the next item a user is likely to interact with, given a short sequence of previous interactions (a session). Traditional approaches such as collaborative filtering or matrix factorization are not well-suited for this problem due to their reliance on long-term user profiles. Likewise, Markov models are limited by their assumption of short-term dependencies and by their computational inefficiency in large state spaces [**?**].

Recent advances in deep learning have opened new possibilities for sequential recommendation. While Recurrent Neural Networks (RNNs), LSTMs, and GRUs have shown significant improvement by modeling sequential dynamics, Graph Neural Networks (GNNs) have emerged as a powerful alternative. GNNs can naturally model sessions as directed graphs where nodes represent items and edges represent transitions between them. Their ability to capture high-order relationships and contextual information from neighboring nodes makes them especially suited for capturing complex session patterns.

In this work, we propose a hybrid GNN-based architecture that incorporates both sequential and non-sequential information. Sessions are modeled as directed graphs to capture the order of interactions, while an auxiliary undirected graph captures item co-occurrence to enrich item embeddings. The model is further enhanced with an attention mechanism that dynamically weighs the importance of items in the session. Experimental evaluations are conducted using publicly available benchmarks such as YooChoose and Diginetica. Preliminary results suggest that integrating structural and contextual signals leads to improved next-item prediction in session-based recommendation.

Acknowledgements

Aquí van los agradecimientos...

ÍNDICE

List of figures

List of tables

CHAPTER 1

INTRODUCTION

Recommender Systems (RS) aim to predict user preferences and suggest items (e.g., products, movies) by leveraging historical interactions. RS analyze user behavior to improve the user experience in wide applications such as social networks, video platforms, entertainment, and e-commerce.

Session-based recommendation (SB-Recommendation) is relatively an unappreciated problem in the machine learning and recommendation systems community [18]. In S-B recommendation, the goal is to predict the next item in a sequence of interactions (e.g the next click in a sequence of clicks). Many e-commerce, news, and media platforms do not reliably track user identities over time, either due to privacy concerns or limitations of cookies and browser fingerprinting. Consequently, in this specific task, each session is often treated independently (as anonymous sessions), without access to a persistent user profile and trusting only on the sequences themselves [18]. Most deployed session-based recommender systems rely on simple methods such as item-to-item similarity, co-occurrence, or transition probabilities. While these approaches can produce acceptable results, they typically consider only the user's last interaction, ignoring the full sequence of past interactions.

1.1 Background and Motivation

RS leverage algorithms to predict user interests, helping them to discover new items while increasing customer satisfaction, engagement, and potentially sales. Recently, due to the large amount of data available on the internet, RS are an essential part of websites such as e-commerce sites and entertainment applications, among others, since they can help alleviate information overload.

Modern recommendation systems use artificial intelligence algorithms, usually associated with

machine learning algorithms. Also, modern RS, use Big Data. These can be based on various criteria, including past purchases, search history, demographic information, and other factors such as semantic information. In addition to sequences, the way interactions between users and items are modeled, allows to build a graph representation naturally.

1.2 Traditional approaches

1.2.0.1 Collaborative Filtering

Collaborative filtering (CF) is a core algorithm widely used in recommended systems. It uses the known preferences of a group of users to make recommendations or predictions of those unknown preferences for other users.[3]

In Collaborative Filtering, there are 2 types of techniques: **Memory-based and Model-based techniques**.

1.2.0.2 Memory Based

This approach uses the entire or a sample of the user-item database to generate a prediction. It considers groups of users. A user belongs to a particular group (neighborhood) based on similarities.
A well-known memory-based algorithm is the neighborhood-based CF algorithm.

1.2.0.3 Model Based

Model-based techniques use pure rating data to train a model to make predictions. The model can be a common Machine Learning algorithm. Well-known model based CF techniques include Bayesian belief networks, clustering models and latent semantic CD models. In addition to collaborative filtering.

All of these methods are not suitable for sequential recommendation since they only focus on user-item interactions rather than sequential modeling.

1.2.1 Factor models and KNN approaches

In earlier years, the most used approaches in recommender systems were factor models [3] [4] [5] [6] and neighborhood methods [7] [8]. However, these methods often rely on the existence of a user's profile. Factor models operate by decomposing a sparse user–item interaction matrix into a set of d-dimensional vectors, one for each user and item in the dataset. The recommendation task is then formulated as a matrix completion (or reconstruction) problem, where the latent factor vectors are leveraged to estimate missing entries, for instance, by computing the dot product

between some user and item latent representations. However, factor models are difficult to apply in session-based recommendation, since explicit user profiles are often unavailable and more importantly, the order of items in sessions is not taken into account under this approach. On the other hand, neighborhood methods, which rely on measuring similarities between items (or users), are based on co-occurrence information within sessions (or user profiles). These methods have been extensively employed in session-based recommendation.

Traditional methods also include collaborative filtering, content-based filtering, and hybrid approaches. These approaches often rely on shallow models such as matrix factorization, which often rely on the existence of user profiles and not less important, they are difficult to apply in this context since they cannot deal with sequential information.

1.2.2 Markov Decision Processes

Other approaches have also shown promising results in session-based recommendation, notably those based on Markov Chains [9, 10]. In this formulation, the recommendation problem is modeled as a Markov Decision Process (MDP). An MDP is defined as a four-tuple $< S, A, Rwd, tr >$. where S is the set of states, A is a set of actions, Rwd is a reward function, and tr is the state-transition function. In particular, the problem of session-based recommendation can be modeled as the simplest MDP. Which is essentially a first-order Markov chain. Here, the next recommendation can be simply computed based on the transition probability between items, and some strong independence assumptions are made: the current event in a session depends only on the immediately preceding event. While Markov-based models can yield competitive results, they face significant scalability challenges when trying to include all possible sequences of user selections. Hence, it becomes computationally expensive, and the resulting state space often grows unmanageably large in real-world scenarios [22].

1.3 Deep Learning Approaches

Recurrent Neural Networks (RNNs) have become the state of the art for sequential recommendation. The evolution of sequential RS has transitioned from Markov Chains and session-based KNN to sophisticated deep learning approaches, including RNNs, LSTMs, attention mechanisms, and transformer architectures [28]. Due to their ability to model sequential data, the authors in [**?**] were among the first to tackle session-based recommendation using recurrent neural networks (RNNs). They propose an end-to-end architecture named GRU4REC that uses a Gated Recurrent Unit (GRU) and adapts two ranking loss functions: The Bayesian Personalized Ranking (BPR), which is a matrix factorization method that uses pairwise ranking loss, and TOP1, which is the regularized approximation of the relative rank of the relevant item.

Another important work based on RNNs is [**?**]. Here, the authors address the limitations of traditional deep learning models by proposing a temporal gating methodology that integrates the time intervals between user interactions. They proposed a Time-aware GRU (a generalization of a traditional GRU) to handle temporal information, and a time-aware attention mechanism to query long-term historical records in a weighted manner, using time as the weight during attention score computation.

The approach using the graph neural network (GNN) framework generally involves building directed graphs for each session and then applying GNNs to learn item embedding representations. The main difference between GNNs and traditional deep learning architectures like [A MLP session-based recommendation] is that GNNs capture the graph's structure via neighborhood aggregation, in sequential recommendation, sessions are modeled as graphs, and GNNs have shown to capture richer embedding representations. [23] was the first work that used GNNs for sequential recommendation. The authors proposed a Gated GNN (GRU-based) to model sequential interactions. The proposed architecture outperformed traditional methods. In 2022, based on [23], Tajuddeen et al. Proposed a similar architecture in [**?**]. The main difference in the architecture is that it takes non-sequential information into account and uses a traditional GNN to first generate embeddings for non-sequential information. Then, the embeddings are fed into the Gated GNN as inputs. Session based recommendation relies on implicit feedback. Since we have it not explicitly such as in other recomendation tasks such in [25–27]

1.4 Problem statement

The problem this work aims to solve is predicting the next item to be visited, given an anonymous session of interactions. Let V be the set of all items and S the set of all sessions in the dataset. A session $s \in S$ is defined as an ordered sequence of items from V i.e. $s = [v_{s1}, v_{s2}, \ldots, v_{sn-1}]$. Where each $v_{si} \in V$. The task is to predict the next item $v_{sn} \in V$ that follows the sequence s.

1.4.1 Objectives

Main objective

The objective of this work is to develop a session-based recommendation system using Graph Neural Networks and Attention Mechanisms, evaluating the impact of sequential and non-sequential information, as well as temporal dynamics, on the recommendation performance.

Specific objectives

- To conduct a review of similar works for sequential recommendation, focusing specifically on approaches leveraging Graph Neural Networks (GNNs) and attention mechanisms, to synthesize key limitations and opportunities in the current research landscape.

- To propose and develop a recommendation architecture, integrating graph neural networks to obtain embedding representations of the nodes from session graphs.
- To incorporate and evaluate the effect of sequential and non-sequential information from the session graph within the proposed GNN framework, and analyze its impact on the final recommendation performance.
- To integrate an attention layer to obtain the final session representation based on attention weights computed for each item present in the session.
- To model temporal dynamics by incorporating time information (e.g., normalized time elapsed between sequential interactions), capturing the importance of some items according to their time span in the attention layer.
- To evaluate the proposed architecture against similar session-based recommendation baselines using standard benchmark datasets, analyzing results obtained in performance metrics, specifically Precision@K and MRR@K.
- To perform ablation studies to validate the contribution of each component: sequential, non-sequential information, and time dynamics.

1.4.2 Common challenges in recommender systems

Data Spartsity
In real-world scenarios, recommendation systems are widely used for commercial purposes, where the set of products can be extremely large. Hence, the user-item matrix used for collaborative filtering could be extremely sparse, which impacts the performance of recommendations.

Cold Start The cold start problem is a challenge to face in sparse matrices. When a new user or item has just entered the system, it is hard to find similar users for that user due to a lack of information about their preferences. In session-based recommendation, the cold start problem is mitigated. Since we don't rely on the existence of a user profile and we only trust on the sequence itself.

CHAPTER 2

FUNDAMENTALS

2.1 Recurrent Neural Networks (RNN)

2.1.1 Traditional RNN

2.1.2 Long Short Term Memory unit (LSTM)

2.1.3 Gated Recurrent Unit (GRU)

2.2 Graph Learning Based Recommender Systems

Graph Learning Based Recommender Systems (GLRS) are build on graphs, where objects such as users or items are implicitly or explicitly connected.
Many Graph Learning (GL) techniques such as random walks and Graph Neural Networks (GNNs) have demonstrated to be quite effective learning the particular type of relations modeled by graphs [Wu et al., 2021]. Therefore, Using GL results as a natural choice in modeling various relations in Recommender Systems.

2.3 Graphs

We define a graph as set containing two sets. Nodes (V) and Edges (E):

$$G = \{V, E\} \tag{2.1}$$

V contains nodes (e.g. items in a session) and E indicate connections between them. A graph can be directed or undirected. In a directed graph, the edges have a specific direction. An edge

from node u to node v is distinct from an edge from node v to node u. Therefore, the edge set E_d consists of ordered pairs of vertices.

$$G_d = \{V, E_d\} \tag{2.2}$$

Where the edge set is defined as a subset of the Cartesian product of V with itself:

$$E_d \subseteq \{(u,v) \mid u,v \in V\} \tag{2.3}$$

Where, (u,v) represents a directed edge that points from node u to node v. $(u,v) \neq (v,u)\ \forall\ u \neq v$. An undirected graph, contains symmetrical connections. If node u is connected to node v, then node v is simultaneously connected to node u. Therefore, the edge set E_u consists of unordered pairs (sets of two distinct elements).

$$G_u = \{V, E_u\} \tag{2.4}$$

The edge set E_u is defined as:

$$E_u \subseteq \{\{u,v\} \mid u,v \in V \text{ and } u \neq v\} \tag{2.5}$$

In practice, graphs can be represented as matrices or adjacency lists. In this work, graphs are represented as matrices. Since we used the framework Pytorch, we take advange of Graphical Processing Units (GPUs) to perform matrix operations efficiently. Figure [a] shows a representation of a session graph as an adjacency matrix.

2.4 The Graph Neural Network Framework

The graph neural network formalism is a general framework for defining deep neural networks on graph data. The primary goal is to generate representations of nodes that actually depend on the structure of the graph. As well as any feature information present in the data. The main challenge is to develop encoders for graph structure data. Since the traditional deep learning toolbox does not work on it. Thus, a new kind of deep learning architecture needs to be defined.[24]

As mentioned earlier, Graph Neural Networks (GNNs) aim to generate representations of nodes. These representations are embeddings of the original nodes. Hence, an embedding space is generated by GNNs, where the main idea is that nodes that are close in the embedding space (using some distance metric such as the dot product) are also close in the original graph.

2.4.1 Defining a Graph Neural Network

A GNN consists of two key components. The aggregate and update functions. Aggregate is defined as follows:

$$m_{N(u)}^{(k)} = AGGREGATE\left(\{h_v^{(k)}, \forall v \in N(u)\}\right) \tag{2.6}$$

As seen in equation 2.6, a set of neighbor feature vectors is taken, $\{h_v^{(k)}, \forall v \in N(u)\}$, and then, combined into a single one, fixed-size vector. This operation acts as a compressed and summarized representation of the node's entire neighborhood. Where $m_{N(u)}^{(k)}$ represents the aggregated "message" from the neighborhood of node v at layer k.

The update function is defined as follows:

$$h_u^{(k+1)} = UPDATE^{(k)}\left(h_u^{(k)}, m_{N(u)}^{(k)}\right) \tag{2.7}$$

The *UPDATE* function is typically a linear transformation transformation followed by a non-linearity (an activation function such as ReLU), typically a neural network with learnable weights. But could be any other arbitrary differentiable function. *UPDATE* provides the embedding's new vector representation $h_u^{(k+1)}$. It takes the result from the *AGGREGATE* function as well as the embedding representation of the previous time $h_u^{(k)}$ as input. Note that if we only used the aggregated neighborhood message, the node would lose its original information, since the update would rely only on the information from the node's neighborhood, loosing the node's identity. By using both, the node representation and its message, the GNN allows the node to both retain its own identity and incorporate information from its local graph structure. The initial embeddings at k=0 are set to the input features for all the nodes. That is: $h_u^{(0)} = \mathbf{x}_u, \forall u \in V$. The final embedding representation for each node is obtained after running K iterations (e.g K layers) of the GNN message passing. The general formula for a single layer in a message passing GNN is defined as follows:

$$h_u^{(k+1)} = UPDATE^k\left(h_u^{(k)}, AGGREGATE\left(h_v^{(k)}, \forall v \in N(u)\right)\right) \tag{2.8}$$

For a node u, its node embedding h_u is transformed as it passes through layer k to produce its output at layer $k+1$. Where:

- h_u^k is the feature vector (or hidden state) of node u at the layer k
- $N(u)$ is the set of all neighbors of node u
- The $AGGREGATE$ function (such as min, max or sum) compresses the feature vectors of all neighbors $\{h_v^{(l)}\}$ into a single vector.
- The $UPDATE$ function, combines the node's own vector $h_v^{(l)}$ with the aggregated neighborhood vector.
- $h_u^{(k+1)}$ is the final output: the new feature vector for node u at the next later, $k+1$

2.4.2 Permutation Invariance Property

Because the neighbors of a node lack a natural or predefined ordering, any neighborhood aggregation function must be permutation invariant. More formally, let $G = (V, E)$ be a graph represented

by an adjacency matrix $A \in \mathbb{R}^{|V|\times|V|}$, and let $X \in \mathbb{R}^{|V|\times d}$ denote the node feature matrix, where d is the input feature dimension. A graph-level function $f : \mathbb{R}^{|V|\times|V|} \times \mathbb{R}^{|V|\times d} \longrightarrow \mathbb{R}^d$ that maps the graph structure and node features to a d-dimensional latent representation is considered permutation invariant if it satisfies:

$$f(A,X) = f(PAP^T, PX) \tag{2.9}$$

where $P \in \{0,1\}^{|V|\times|V|}$ is an arbitrary permutation matrix. Essentially, permutation invariance guarantees that the model's output is independent of the arbitrary indexing of the nodes in the graph representation.

2.4.3 Permutation Equivariance Property

While graph-level representations must be invariant to node ordering, functions operating at the node level—such as those mapping input features to node-specific latent embeddings—must satisfy the property of permutation equivariance. Formally, a node-level function $f : \mathbb{R}^{|V|\times|V|} \times \mathbb{R}^{|V|\times d} \longrightarrow \mathbb{R}^{|V|\times d'}$ maps the adjacency matrix A and node feature matrix X to an updated node representation of dimension d'. The function f is said to be permutation equivariant if, for any permutation matrix $P \in \{0,1\}^{|V|\times|V|}$, it satisfies:

$$f(PAP^T, PX) = Pf(A,X) \tag{2.10}$$

In other words, applying a structural permutation to the input graph and subsequently processing it yields the exact same node embeddings as processing the original graph and then permuting the output rows. This property guarantees that the learned representation of a specific node depends strictly on its structural position and local neighborhood within the graph, rather than its arbitrary index in the data matrix.

The structure of a graph encodes a significant amount of implicit information, which is often hard to capture using traditional learning techniques. By representing data as a graph, these hidden relationships become explicitly modeled. This is where Graph Neural Networks (GNNs) come in. They are designed to take advantage of the relational structure of the data to enhance performance on tasks such as rating prediction or similar content retrieval, which are common in recommender systems.

The primary goal of GNNs is to learn better representations (embeddings) of nodes by aggregating information from their neighbors. Although GNNs can also be applied to learn representations for edges or even entire graphs, this work focuses specifically on node-level embeddings.

In practice, GNNs iteratively update node representations by aggregating feature information from neighboring nodes. The aggregation function defines how the features from neighbors are combined. Common strategies include mean pooling, max pooling, and others. These strategies should be permutation invariant operations [5].

2.4.4 The basic GNN

We begin here with the most basic GNN framework, which is a simplification of the original GNN models proposed by Merkwirth and Lengauer [2005] and Scarselli et al. [2009]. The basic GNN message passing is defined as:

$$h_u^k = \sigma\left(W^{(h)} h_u^{(k-1)} + W_{neigh}^{(k)} \sum_{v \in N(u)} h_u^{(k-1)} + b^{(k)}\right) \tag{2.11}$$

Where:

h_u^k is the embedded vector u at the $k-th$ iteration (or layer)

$h_u^{(k-1)}$ is the embedded vector u at the previous time

$b^{(k)} \in \mathbb{R}^{d^{(k)}}$ is the bias vector (Often omitted for notational simplicity)

$W_{self}^{(k)}, W_{neigh}^{(k)} \in \mathbb{R}^{d^{(k)} \times d^{(k-1)}}$

σ represents a non-linearity (such as tanh or RELU)

First, we compute the sum over all u's neighbors at the previous layer. Then, we combine the neighborhood information with the node's previous embedding using a linear combination; and finally, we apply an elementwise non-linearity

2.4.5 Graph Convolutional Neural Network (GCNN)

One of the most popular baseline graph neural network models—the graph convolutional network (GCN)—employs the symmetric-normalized aggregation as well as the self-loop update approach. The GCN model thus defines the message passing function as:

$$\mathbf{h}_u^{(k)} = \sigma\left(\mathbf{W}^{(k)} \sum_{v \in N(u) \cup \{u\}} \frac{\mathbf{h}_v}{\sqrt{|N(u)||N(v)|}}\right) \tag{2.12}$$

$y = f(X, \ldots)$ **Set Pooling**
In 2017, [29] showed that t an aggregation function with the following form is a universal set function approximator: [24]

$$\mathbf{m}_{N(u)} = MLP_\theta\left(\sum_{v \in N(u)} MLP\phi(\mathbf{h}_v)\right) \tag{2.13}$$

Where usually, MLP_θ represents a deep multilayer perceptron, parametrized by some learnable parameters θ. According to [24]. Any permutation-invariant function that maps a set of embeddings to a single one.

2.4.6 Gated Graph Neural Network (GGNN)

2.4.7 Popular GNN Architectures

GNN Type	**Mechanism**
Graph Convolutional Networks (GCNs)	Aggregate features from neighboring nodes using convolutional operations.[4]
Graph Attention Networks (GATs)	Weight neighbor contributions via attention mechanisms.[5]
Gated Graph Sequence Neural Networks (GGNN)	adopts a gated recurrent unit (GRU).[6]
Hypergraph Neural Networks (HGNN)	is a typical hypergraph neural network, which encodes high-order data correlation in a hypergraph structure[6]
GraphSAGE	samples a fixed size of neighborhood for each node, proposes mean/sum/maxpooling aggregator and adopts concatenation operation for update [6]
Recurrent GNNs	Model sequential interactions (e.g., session-based recommendations). [5]

Table 2.1: GNN Key architectures

2.4.8 Challenges in GNN-Based Recommender Systems

- Graph Construction: How to define meaningful edges and weights.
- Handling heterogeneous graphs with multiple types of nodes/edges.
- Scalability: Large-scale graphs (e.g., billions of nodes/edges) pose memory and computational challenges.
- Model Optimization: Balancing over-smoothing (loss of node-specific information) with under-smoothing (insufficient propagation).
- Efficient training strategies like mini-batch training.

CHAPTER 3

RELATED WORK

In 2019, Wu et al. [**?**] proposed the use of Graph Neural Networks (GNNs) for Session-based Recommendation to address the limitations of conventional sequential models, such as RNNs [cite here RNN-based work], which struggle to capture complex item transitions beyond simple linear ordering. The authors hypothesized that GNNs can take advantage of structural dependencies by modeling sessions as directed graphs (session graphs) where items act as nodes and edges represent sequential transitions. They proposed SR-GNN, an architecture where node embeddings are learned by propagating information across the session graph using a variant of a GNN called Gated GNN where the update operation is a GRU-based function such as in equation [GRU-Equation]. Once the item embeddings are obtained from the GNN, the model generates a comprehensive session representation by dynamically balancing a user's long-term and short-term preferences.The user's short-term interest, or local embedding, is represented directly by the latent vector of the last clicked item in the session: $s_{local} = v_n$. In order to capture the long-term preference, or global embedding, a soft-attention mechanism is applied over all items involved in the session. The attention weight α_i for each item v_i is computed by measuring its relevance against the final item (acting as a query vector) v_n:

$$\alpha_i = q^T \sigma(W_1 v_n + W_2 v_i + c) \tag{3.1}$$

where $W_1, W_2 \in \mathbb{R}^{d \times d}$ and $q, c \in \mathbb{R}^d$ are trainable parameters, and σ is a non-linear activation function. The global session representation $s_{global} \in \mathbb{R}^d$ is subsequently formulated as a weighted sum of the item embeddings:

$$s_{global} = \sum_{i=1}^{n} \alpha_i v_i \tag{3.2}$$

Finally, the hybrid session representation s_h is constructed by applying a linear transformation over the concatenation of the local and global representations:

$$s_f = W_3[s_{local}; s_{global}] \tag{3.3}$$

where $W_3 \in \mathbb{R}^{d \times 2d}$ compresses the combined embeddings back into the d-dimensional latent space for next-item prediction.

In 2020, authors in [**?**] proposed a novel Target Attentive Graph Neural Network (TAGNN) for session-based recommendation. They hypothesized that candidate items are usually abundant and user's interests are usually diverse. So, representing the whole session as a single embedding vector would not be the best option. The proposed model aims to adaptively activate user interests by considering the relevance of historical behaviors given a target item. They introduced a local target attentive layer, where specific user interests related to a target item in the current session are activated, which will enhance the final session representation. Here, the target items as all candidate items to predict. The target attentive layer consist of a novel proposed attention mechanism. It computes attention scores between all items v_i in session s and each target item $v_t \in V$. Firstly, a shared non-linear transformation parameterized by a weight matrix $W \in R^{d\ddot{O}d}$ is applied to every node-target pair. Then, the self-attention scores are normalized using the softmax function:

$$\beta = softmax(e_i, t) = \frac{exp(v_t^T W v_i)}{\sum_{j=1}^{m} exp(v_t^T W v_j)} \tag{3.4}$$

The representation for each session based on a target item v_t is represented by $s_{target}^t \in \mathbb{R}^d$ and is computed as follows:

$$s_{target}^t = \sum_{i=1}^{s_n} \beta_{i,t} v_i \tag{3.5}$$

DINSEN-GNN [**?**] was proposed in 2022, authors mentioned that initializing item embeddings uniformly under the assumption that an item possesses a single, static representation is not the best approach. They argued that user's behavior during a session is dynamic and the motivation for clicking an item is driven by diverse latent factors (e.g., brand, price, style). Consequently, two items might be highly similar regarding one factor but completely different regarding another. Here, they proposed to represent the items in the latent space as features of different factors. They refered to the disentangled learning techniques to learn the item embeddings, in which the representation of each item is formed by K chunks. Each chunk representing a particular factor and those factors being independent of each other. Chunks (factors) are built using the following mechanism:

$$\boldsymbol{c}_{i,k} = \frac{\sigma(\boldsymbol{W}_k^\top \cdot \boldsymbol{e}_{s,i}) + \boldsymbol{b}_k}{||\sigma(W_k^\top \cdot \boldsymbol{e}_{s,i}) + \boldsymbol{b}_k||_2} \tag{3.6}$$

Where $\boldsymbol{W}_k \in \mathbb{R}^{d \times \frac{d}{K}}$ and $\boldsymbol{b}_k \in \mathbb{R}^{\frac{d}{K}}$ are the parameters of the K^{th} factor. And $\boldsymbol{e}_{s,i}$ is the embedding of item i in the session s From chunks, the initial embedding for session s is represented as

$\boldsymbol{E}_s^{(0)} = \{\boldsymbol{e}_{s,1}^{(0)}, \boldsymbol{e}_{s,2}^{(0)}, \ldots, \boldsymbol{e}_{s,n}^{(0)}\}$, where $\boldsymbol{e}_{s,i}^{(0)} = [\boldsymbol{c}_{i,1}^{(0)}, \boldsymbol{c}_{i,2}^{(0)}, \cdots, \boldsymbol{c}_{i,k}^{(0)}]$. Basically, each item is represented by concatenating the features from different factors as the previous equations described. Authors mentioned that two items can be similar on one factor, but different on another. Therefore, they defined a factor-based similarity matrix to compute the similarities between adjacent items for each factor. For a given session s, the similarity between two adjacent items i and j at factor k is computed as: $w_{i,j}^{k} = \boldsymbol{c}_{i,k}^{\intercal} \cdot \boldsymbol{c}_{j,k}$ These weights are then used to build an adjacency matrix for each session graph, which is fed into the Graph Neural Network similar to the one in [**?**], but with modifications For instance, a distance correlation function is used as a regularizer on the first layer of the GNN to measure and encourage independence between the factors of the items in the session The authors also proposed a residual attention mechanism to alleviate the problem of over-smoothing in session-based recommendation by helping items retain their unique features. For the final session embedding learning, the model combines the user's local preference and global preference to accurately capture dynamic and noisy user intents. The local intent s_l is inferred directly from the last clicked item in the session, such that s_l =e s,n

CHAPTER 4

SOLUTION APPROACH

4.1 Notation

Scalars are written in lower-case letters, vectors in lower-case letters in bold, matrices in upper-case in bold and sets in upper-case letters. Table 4.1 summarizes the notation used in this work.

Expression	Meaning
V	Set of all items
$\boldsymbol{v_{si}}$	i-th item from session s
S	Set of all sessions
$\boldsymbol{s}$	Anonymous session
$\boldsymbol{G_s}$	Graph of session s
$\boldsymbol{A}$	Adjacency Matrix
$\boldsymbol{A_{in}}$	Adjacency matrix (incoming links)
$\boldsymbol{A_{out}}$	Adjacency matrix (outgoing links)
$\boldsymbol{W}$	Transformation matrix
$\boldsymbol{b}$	Bias vector
z_s^t	Update gate of session s at time t
r_s^t	Reset gate of session s at time t

4.2 Data

The proposed model was evaluated on three representative real-world datasets for session-based recommendation: **Diginetica**, **Yoochoose 1/64** (a standard fraction of the massive Yoochoose dataset) and Nowplaying. The first two contain information from e-commerce, while the last

one comes from the domain of music. Diginetica is a retail technology company. The Diginetica dataset is from the CIKM 2016 Cup and is publicly available at this link. It has 573,935 sessions, 134,319,529 products, 18,025 purchases, and contains TimeStamp information between interactions. The Yoochoose dataset contains information on six months of activity from a large e-commerce business in Europe selling a wide range of products, including garden tools, toys, clothes, electronics, and more. The dataset is from the RecSys Challenge 2015, it contains a collection of sequences of click events, click sessions, and purchase events. It has 9,249,729 sessions, 52,739 products, 1,150,753 purchases, and TimeStamp information. For a fair comparison with other baselines, for all datasets, sessions of length 1 and items appearing fewer than 5 times are filtered out. To allow our model to potentially generalize better, data augmentation is performed, generating sets of sub-sessions. Specifically, for a session $s = [v_{s1}, v_{s2}, \ldots, v_{sn}] \in S$, multiple sequence-target pairs are created as follows: $\{([v_{s1}, \ldots, v_{si}], v_{s,i+1}) \mid 1 \leq i < n\}$. The final statistics of the 3 datasets after pre-procesing are summarized in table 4.2:

Table 4.1: Statistics of Diginetica, Yoochoose and Nowplaying datasets after preprocessing

Statistics	Yoochoose 1/64	Diginetica	Nowplaying
# Clicks	557,248	982,961	1,367,963
# Training sessions	369,859	719,470	825,304
# Test sessions	55,898	60,858	89,824
# Items	16,766	43,097	60,417
Avg. length	6.16	5.12	7.42
Max length	146	70	30

An end-to-end architecture is proposed based on graph neural networks and an attention mechanism.

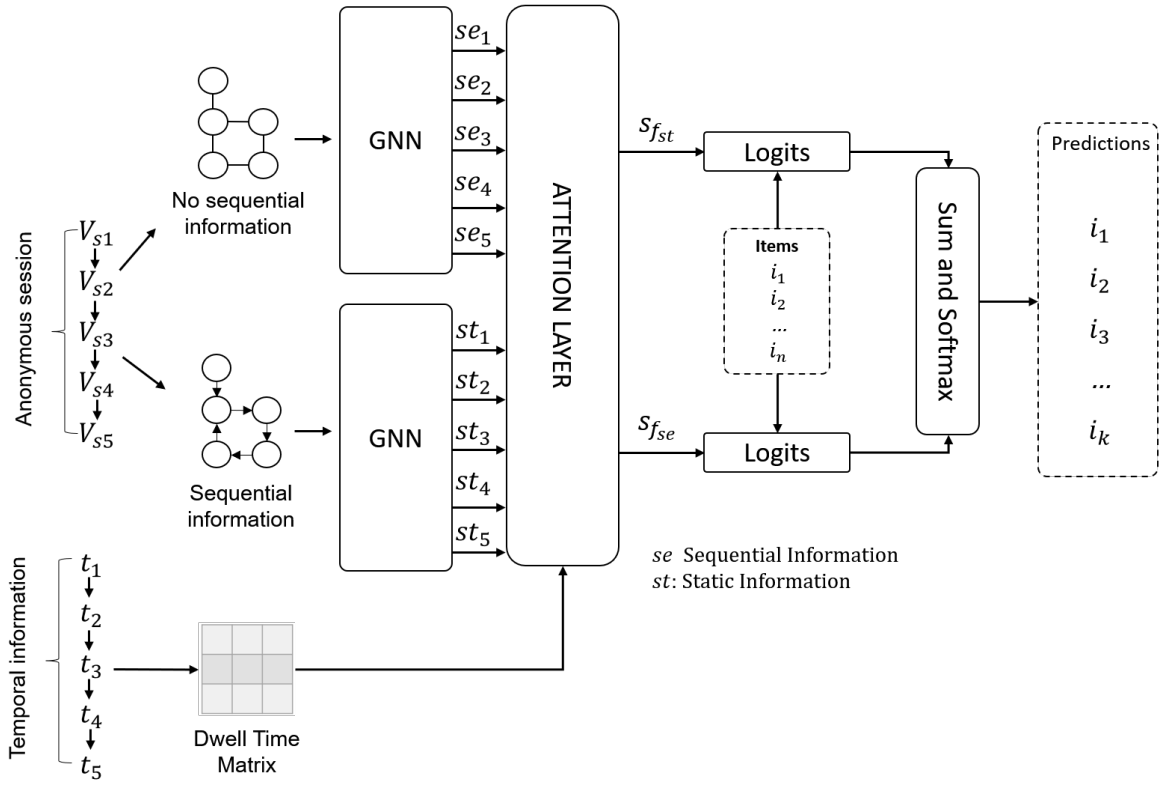

Figure 4.1: High-level diagram of the proposed architecture

For sequential modeling, a gated graph neural network is proposed, similar to [23] and [30]. As mentioned in equations [cite theory], message and update functions are proposed as follows:

4.3 Sequential Modeling

Message function:

$$a_s = [A_{in} h_{s-1} W_{in}, A_{out} h_{s-1} W_{out}] \tag{4.1}$$

The message is obtained by multiplying each node's neighbor's embedding by the node's own embedding. The product of matrices A_{in} and A_{out} with node vectors h_{s-1} is a permutation invariant operation.

Update:

$$z_s = \sigma([a_s, h_{s-1}, W_z + b_z]) \tag{4.2}$$

$$r_s = \sigma([a_s, h_{s-1}, W_r + b_r]) \tag{4.3}$$

$$\tilde{hs} = tanh([a_s, (r_s \odot h_{s-1}) W_h + b_h]) \tag{4.4}$$

$$h_s = z_s \odot h_{s-1} + (1 - z_s) \odot \tilde{h_s} \tag{4.5}$$

4.4 Non-sequential Modeling

We define sequential modeling as the structural information without taking into consideration the order of the items in the session. Similarly to sequential modeling, we use a graph neural network framework to obtain node embeddings for each node in the session. For the update, we propose a slightly different function, following the gating idea, the message and update functions are given as follows:
Message function:

$$a_s = [Ah_{s-1}] \tag{4.6}$$

Update function:

$$\hat{a_s} = LeakyReLU(W_s a_s) \tag{4.7}$$

$$g_1 = \sigma(W[\hat{a_s}, h_{s-1}]) \tag{4.8}$$

$$h_s = g_1 \cdot h_{s-1} + (1 - g_1) \cdot h_{s-1} \tag{4.9}$$

4.5 Attention Layer

Once the embeddings for session items are computed via Graph Neural Networks, an attention mechanism is used to capture the importance of all items in the sessions. And then, the final session representation is obtained by computing a compact representation of the whole session by simply adding up all item embeddings. Similar to [31], scaled-dot-product attention is used:

$$Scores = \frac{QK^T}{\sqrt{d}} \tag{4.10}$$

Where $Q = W_q H$ and $K = W_k H$, being H the item embeddings of the current session. W_Q and W_K are transformation matrices to project current embeddings in the latent space. The scores are divided by $\sqrt{d}$ to avoid large values of the inner product, especially when d (the size of the hidden dimension of the vectors in the latent space) is high.

Time aware component
In order to let the attention mechanism be time-aware using the temporal information in the dataset, similar to [30], the idea is to take the time component into account when computing the correlation between 2 items. First, a time-interval matrix is computed and defined as follows:

$$Scores = \frac{QK^T \odot G}{\sqrt{d}} \tag{4.11}$$

A matrix T is defined as $T = H \hat{-} H$. $H \in \mathbb{R}^{l \times d}$ Such that $T_{i,j} = |H_i - H_j|$ and $T \in \mathbb{R}^{n \times n}$. A time gate is computed as follows:

$$\delta = \phi(log(T) + 1) \odot W_\delta + b_\delta \tag{4.12}$$

$$\tau = \phi(W_t h + b_\tau) \tag{4.13}$$

$$G = \sigma(\delta \odot W_{G_\delta} + \tau \odot W_{G\tau} + b_g) \tag{4.14}$$

The intuition of the time gate G is to somehow learn the importance between temporal and structural information. δ captures the importance of the temporal information for each pair of correlated items according to their time span in the session. Similarly, τ captures the structural information of the session. Finally, the gate G, computes the final trade-off between temporal and structural information using a linear interpolation between δ and τ. Once the time gate G is computed, we can update the attention scores described in equation 4.10 as follows:

$$Scores = Softmax\left(\frac{QK^T \odot G}{\sqrt{d}}\right) \tag{4.15}$$

Where $Q, K \in \mathbb{R}^{n \times d}$ and $G \in \mathbb{R}^{n \times n}$. Then, the final attention is computed using equation 4.15 as follows:

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T \odot G}{\sqrt{d}}\right)V \tag{4.16}$$

Where $V \in \mathbb{R}^{n \times d}$ is simply the vector H of the items in the session and $Attention(Q, K, V) \in \mathbb{R}^{n \times d}$ contains the weights for all items in the session. From equation 4.16, the final session representation s_g is obtained:

$$s_g = \tag{4.17}$$

4.6 Prediction Layer

Using the final representations of the sessions $s_{f_{se}}$ and $s_{f_{st}}$, the final prediction is obtained. From each session representation, item logits are computed as follows:

$$logits_{se}(V, s_{f_{se}}) = V s_{f_{se}} \tag{4.18}$$

$$logits_{st}(V, s_{f_{st}}) = V s_{f_{st}} \tag{4.19}$$

Where $s_{f_{se}}$ and $s_{f_{st}} \in \mathbb{R}^d$ and $V \in \mathbb{R}^{n \times d}$. To obtain the final item scores, logits are added up and softmax function is applied:

$$scores = Softmax(logits_{se} + logits_{st}) \tag{4.20}$$

Finally, for the final recommendation, $scores$ are sorted and the first top K items are selected.

4.6.1 Model Evaluation

For evaluation, 2 metrics were considered: *Precision@K* (also known as *Hit Rate*) and Mean Reciprocal Rank *MRR@K*.

Precision@K is defined as follows:

$$Precision@K = \frac{\lambda_{hit}}{N} \tag{4.21}$$

This metric captures model precision by counting the number of times the target element was within the *TOP K* recommendations. λ_{hit} is the number of times the prediction was within the top 20 recommendations among the whole batch of sessions. Being N the number of sessions in the batch. *MRR*@20 is defined as follows:

$$RR@K = \frac{1}{rank(i)} \tag{4.22}$$

$$MRR@K = \frac{1}{N} \sum_{i \in Rec}^{N} RR@K \tag{4.23}$$

MRR@20 evaluates the quality of the recommendation by taking into account the position of the recommendation. i represents the target item. And *rank(i)* tells us the position of the item in the *TOP K* recommendations. *RR@K* is the reciprocal of the rank. And *MRR*@K is the mean of the reciprocal rank across N different sessions. If the target item is not in the recommendation list, $RR@K = 0$.

4.6.2 Loss function

For any given session, the categorical cross-entropy loss function between the predictions and the ground truth is used:

$$L(\hat{\mathbf{y}}) = -\sum_{i=1}^{|V|} y_i \log(\hat{y}_i) \tag{4.24}$$

Where $\mathbf{y}$ denotes the ground truth one-hot vector, $\hat{\mathbf{y}}$ is the vector of predicted probabilities for all V items and $\hat{y}_i$ is the probability assigned to item i. Since the ground truth for each session is a one-hot encoded vector all elements y_i are zero except for the one corresponding to the actual next item in the session ($y_j = 1$). Consequently, Equation 4.24 simplifies to the negative log-likelihood of the true item:

$$L(\hat{\mathbf{y}}) = -\log(\hat{y}_j) \tag{4.25}$$

where $\hat{y}_j$ represents the predicted probability for the ground truth item j.

4.7 Research Questions (RQ)

We propose the following research questions:

- RQ1: Can the proposed architecture achieve state of the art results?

- RQ2: What is the effect of sequential and non-sequential information on the recommendation performance?
- RQ2: What is the effect of the attention layer in the final recommendation?
- RQ3: What is the effect of the local and global representation of the session?
- RQ4: What is the effect of temporal dynamics in the final recommendation? Does the effect depend of the nature of the temporal information in the data?
- RQ5: What is the performance of the model for short and long sessions?
- RQ6: What is the performance of the model for different top-k recommendations?

CHAPTER 5

EXPERIMENTAL RESULTS

5.1 Experimental setup

For all the experiments, a hidden size $d = 100$ (representing the dimension of item embeddings in the latent space) and a batch size of 100 (e.g., 100 sessions per batch) were used. An initial learning rate (lr) of 0.001 with learning rate decay of 0.1 every 3 steps was used. l2 penalty $l2 = 1 \times 10^{-5}$ was used as a regularization term to prevent overfitting and potentially improve generalization. The number of epochs was set to 30 and a patience parameter was used as an early stopping criterion.
For evaluation, *Precision@20* (also known as *Hit Rate*) and Mean Reciprocal Rank *MRR*@20 were considered. All the parameters of the model were normalized with a mean of 0 and a standard deviation of 0.1. All the experiments were executed 3 times with 3 different seeds (from 40 to 42) and the mean of the results is reported.

5.2 Results obtained

Table 5.1: Combined results for Diginetica, Yoochoose, and Nowplaying datasets

	Diginetica		Yoochoose		Nowplaying
Description	P@20	MRR@20	P@20	MRR@20	MRR@20
Hybrid	53.0338	18.5841	71.6436	31.1399	8.0564
Hybrid + Time Gate	53.1209	18.6351	71.5750	31.0830	8.1571
Only Sequential	52.5497	18.4186	71.2160	30.3928	7.6119
Only Sequential + Time Gate	52.4763	18.3627	71.1630	30.2838	7.5357
Only Non-Sequential	51.8798	17.4643	71.1254	31.2610	8.0580
Only Non-Sequential + Time Gate	51.8913	17.5536	71.1266	31.1471	8.2062

Table 5.2: Short and Long Sessions

Experiment ID	**Description**	**P@20**	**MRR@20**
short_now_a	Hybrid	21.3647	8.0564
long_now_a	Hybrid	21.3198	8.0681
short_dig_a	Hybrid	52.3956	18.8165
long_dig_a	Hybrid	42.8276	14.3372
short_yoo_a	Hybrid	To Be Defined	To Be Defined
long_yoo_a	Hybrid	To Be Defined	To Be Defined

5.3 Comparison with some baselines

To compare the results obtained with baseline algorithms, we select experiments 4 and 7 for the Diginetica and Yoochoose datasets, respectively. We considered the following baseline algorithms for comparison:

- **POP:** This method recommends the most popular item in the dataset.
- **S-POP:** Similar to POP, this method is a session-based algorithm that recommends the most popular item in the session.
- **item-KNN:** This method uses metrics (e.g., cosine similarity) to find and recommend items similar to the last in the session.
- **BPR-MF:** It is a matrix factorization (MF) method. Although MF is not used for session-based recommendation, item embeddings within a session can be learned via a pairwise ranking loss, thereby optimizing the latent representation of items for recommendation.
- **FPMC:** It is an MC-based method for sequential recommendation and next-basket-prediction.

- **GRU4REC:** It is a RNN-based method. It uses a GRU model and a pairwise ranking loss function to perform recommendations.

- **NARM:** It is a transformer-based model. It uses an encoder-decoder architecture for sequence-aware recommendation [**?**]

- **STAMP:** This uses an attention-based model. It tries to capture both. The short and long-term preferences in a sequence for recommendation.

Table 5.3 shows the results comparing experiments 4 and 7 with the baseline methods mentioned above (Best results in bold).

Table 5.3: Performance Comparison Across Different Baselines and Datasets

Type	**Method**	**Diginetica**		**Yoochoose**		**Nowplaying**	
		P@20	MRR@20	P@20	MRR@20	P@20	MRR@20
Shallow	POP	0.89	0.20	6.71	1.65	-	-
	S-POP	21.06	13.68	30.44	18.35	-	-
	Item-KNN	35.75	11.57	51.60	21.81	15.94	4.91
	FPMC	26.53	6.95	45.62	15.01	7.36	2.82
	GRU4REC	29.45	8.33	60.64	22.89	7.92	4.48
RNN	NARM	49.70	16.17	68.32	28.63	18.59	6.93
	STAMP	45.64	14.32	68.74	29.67	17.66	6.88
	SR-GNN	50.73	17.59	70.57	30.94	18.87	7.47
GNN	GC-SAN	50.80	16.87	70.89	30.98	-	-
	FGNN	51.36	18.47	71.12	31.68	18.78	7.15
	OURS	53.05	18.60	71.64	31.26	21.36	8.06

5.4 Comparisson with some recent methods

Table 5.4: Performance Comparison with some Recent Methods (2020–2025)

Year	**Method**	**Diginetica**		**Yoochoose**		**Nowplaying**	
		P@20	MRR@20	P@20	MRR@20	P@20	MRR@20
2020	TAGNN	51.31	18.03	71.02	31.12	-	-
	SGINM	53.91	18.59	70.89	31.33	-	-
2021	MoSeR	52.98	19.14	71.20	31.68	-	-
2022	SGNN	50.91	17.06	70.21	30.34	-	-
	SEDGN	51.91	18.22	71.42	31.15	-	-
	CORE	52.89	18.58	64.61	28.24	21.81	7.35
	SR-HGNN	52.02	18.48	71.18	31.70	-	-
	Disen-GNN	53.79	18.99	71.46	31.36	22.22	8.22
2023	Transformer4Rec	53.32	18.00	70.32	29.63	-	-
2024	TRHMCI-GNN	54.54	18.79	71.91	31.11	-	-
2025	MSERS	56.32	21.62	71.24	31.73	-	-
	GAN-GSNP	50.92	17.51	70.58	31.11	-	-
2026	OURS	53.05	18.60	71.64	31.26	21.36	8.06

CHAPTER 6

CONCLUSIONS AND FUTURE WORK

Las conclusiones y el trabajo a futuro inician aquí...

APPENDIX

Análisis

El apéndice inicia aquí.

References

[1] Su, Xiaoyuan, Khoshgoftaar, Taghi M., A Survey of Collaborative Filtering Techniques, Advances in Artificial Intelligence, 2009, 421425, 19 pages, 2009. https://doi.org/10.1155/2009/421425

[2] Ruslan Salakhutdinov and Andriy Mnih. (2008). Bayesian probabilistic matrix factorization using markov chain Monte Carlo. In Proceedings of the 25th International Conference on Machine Learning

[3] Francisco J. Peña, DIarmuid O'Reilly-Morgan, Elias Z. Tragos, Neil Hurley, Erika Duriakova, Barry Smyth, and Aonghus Lawlor. (2020). Combining Rating and Review Data by Initializing Latent Factor Models with Topic Models for Top-N Recommendation. In RecSys 2020 - 14th ACM Conference on Recommender Systems.

[4] Koren, Yehuda, Bell, Robert, and Volinsky, Chris. (2009). Matrix factorization techniques for recommender systems. Computer, 42(8):30–37.

[5] Weimer, Markus, Karatzoglou, Alexandros, Le, Quoc Viet, and Smola, Alex. (2007). Maximum margin matrix factorization for collaborative ranking. Advances in neural information processing systems.

[6] Hidasi, B. and Tikk, D. (2012). Fast ALS-based tensor factorization for context-aware recommendation from implicit feedback. In ECML-PKDD'12, Part II, number 7524 in LNCS, pp. 67–82. Springer.

[7] Sarwar, Badrul, Karypis, George, Konstan, Joseph, and Riedl, John. (2001). Item-based collaborative filtering recommendation algorithms. In Proceedings of the 10th international conference on World Wide Web, pp. 285–295. ACM.

[8] Koren, Y. (2008). Factorization meets the neighborhood: a multifaceted collaborative filtering model. In SIGKDD'08: ACM Int. Conf. on Knowledge Discovery and Data Mining, pp. 426–434.

[9] Shani, G.; Brafman, R. I.; and Heckerman, D. (2002). An mdp-based recommender system. In UAI, 453–460.

[10] Rendle, S.; Freudenthaler, C.; and Schmidt-Thieme, L. (2010). Factorizing personalized markov chains for next-basket recommendation. In WWW, 811–820. ACM.

[11] Tan, Y. K.; Xu, X.; and Liu, Y. (2016). Improved recurrent neural networks for session-based recommendations. In DLRS, 17–22.

[12] Tuan, T. X., and Phuong, T. M. (2017). 3d convolutional networks for session-based recommendation with content features. In RecSys, 138–146.

[13] Li, J.; Ren, P.; Chen, Z.; Ren, Z.; Lian, T.; and Ma, J. (2017). Neural attentive session-based recommendation. In CIKM, 1419–1428.

[14] Shoujin Wang and Liang Hu and Yan Wang and Xiangnan He and Quan Z. Sheng and Mehmet A. Orgun and Longbing Cao and Francesco Ricci and Philip S. Yu, (2021), Graph Learning based Recommender Systems: A Review, https://doi.org/10.48550/arXiv.2105.06339

[15] Sanchez-Lengeling, et al., "A Gentle Introduction to Graph Neural Networks", Distill, 2021.

[16] Alexander S. Gillis, What are graph neural networks (GNNs)?, https://www.techtarget.com/searchenterpriseai/definition/graph-neural-networks-GNNs

[17] Santhosh Rajamanickam, (2021), Graph Neural Network (GNN) Architectures for Recommendation Systems, https://towardsdatascience.com/graph-neural-network-gnn-architectures-for-recommendation-systems-7b9dd0de0856/?gi=f357864deb6a

[18] Hidasi, B., Karatzoglou, A., Baltrunas, L., Tikk, D. (2015). *Session-based recommendations with recurrent neural networks.*, arXiv preprint, https://doi.org/10.48550/arXiv.1511.06939

[19] Tajuddeen Rabiu Gwadabe and Ying Liu, *Improving graph neural network for session-based recommendation system via non-sequential interactions*, Neurocomputing, 2022. https://doi.org/10.1016/j.neucom.2021.10.034

[20] Chhotelal Kumar, Mukesh Kumar, (2024), *Session-based recommendations with sequential context using attention-driven LSTM*, Computers and Electrical Engineering, Volume 115, 109138, ISSN 0045-7906, https://doi.org/10.1016/j.compeleceng.2024.109138.

[21] Ehsan Elahi, Sajid Anwar, Mousa Al-kfairy, Joel J.P.C. Rodrigues, Alladoumbaye Ngueilbaye, Zahid Halim, Muhammad Waqas, (2025), *Graph attention-based neural collaborative filtering for item-specific recommendation system using knowledge graph*, Expert Systems with Applications, Volume 266, 126133, ISSN 0957-4174, https://www.sciencedirect.com/science/article/pii/S0957417424030008.

[22] Hidasi, B., Karatzoglou, A., Baltrunas, L., Tikk, D. (2015). *Session-based Recommendations with Recurrent Neural Networks.*, arXiv, https://doi.org/10.48550/arXiv.1511.06939

[23] Wu, Shu and Tang, Yuyuan and Zhu, Yanqiao and Wang, Liang and Xie, Xing and Tan, Tieniu, (2018), *Session-Based Recommendation with Graph Neural Networks*, Proceedings of the AAAI Conference on Artificial Intelligence, Volume 33, ISSN2159-5399, http://dx.doi.org/10.1609/aaai.v33i01.3301346

[24] William L. Hamilton. (2020). Graph Representation Learning. Synthesis Lectures on Artificial Intelligence and Machine Learning, Vol. 14, No. 3 , Pages 1-159.

[25] R. Salakhutdinov, A. Mnih, (2007). *Probabilistic matrix factorization*, Advances in Neural Information Processing Systems, Volume 20, ISN 1257-1264, https://proceedings.neurips.cc/paper$_files/paper$/2007/$file/d7322ed717dedf1eb4e6e52a37ea7bcd-Paper.pdf$

[26] Y. Koren, R. Bell and C. Volinsky, (2009). *Matrix Factorization Techniques for Recommender Systems*, Computer, vol. 42, no. 8, pp. 30-37, 10.1109/MC.2009.263

[27] Zhang, Liang Agarwal, Deepak Chen, Bee-Chung. (2011). Generalizing matrix factorization through flexible regression priors. RecSys'11 - Proceedings of the 5th ACM Conference on Recommender Systems. 13-20. 10.1145/2043932.2043940.

[28] Shaina Raza and Mizanur Rahman and Safiullah Kamawal and Armin Toroghi and Ananya Raval and Farshad Navah and Amirmohammad Kazemeini. (2025). *A Comprehensive Review of Recommender Systems: Transitioning from Theory to Practice*, arXiv, arXiv:2407.13699

[29] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. Smola. (2017). Deep sets. NeurIPS, https://arxiv.org/abs/1703.06114

[30] W Ji, K. Wang, X. Wang, T. Chen, A. Cristea, (2017). Sequential Recommender via Time-aware Attentive Memory Network, ArXiv, https://arxiv.org/abs/2005.08598.

[31] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., Polosukhin, I. (2017). Attention is all you need. arXiv. https://doi.org/10.48550/arXiv.1706.03762

[32] P. Veli ˇckovi ´c, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio. Graph attention networks. (2018), ICLR.